University of Glasgow | School of Computing Science

Assessed Coursework

Course Name	Programming COMPSCI4039			
Coursework Number	1/1			
Deadline	Time:	16h30	Date:	20/11/2026
% Contribution to final course mark	8%			
Solo or Group ✓	Solo	✓	Group	
Anticipated Hours	Average 40 hours			
Submission Instructions	Submission of all of the code needed to run the game.			
Please Note: This Coursework cannot be Re-Assessed				

Code of Assessment Rules for Coursework Submission

Deadlines for the submission of coursework which is to be formally assessed will be published in course documentation, and work which is submitted later than the deadline will be subject to penalty as set out below.

The primary grade and secondary band awarded for coursework which is submitted after the published deadline will be calculated as follows:

(i) in respect of work submitted not more than five working days after the deadline
   a. the work will be assessed in the usual way;
   b. the primary grade and secondary band so determined will then be reduced by two secondary bands for each working day (or part of a working day) the work was submitted late.

(ii) work submitted more than five working days after the deadline will be awarded Grade H.

Penalties for late submission of coursework will not be imposed if good cause is established for the late submission. You should submit documents supporting good cause via MyCampus.

Penalty for non-adherence to Submission Instructions is: 2 bands

COMPSCI 4039 – Programming (IT)

Assessed Exercise – Ecosystem Saviour

1. Introduction – why are you doing this? What is it part of?

This exercise aims to allow you to demonstrate your developing skills in understanding and developing programs, using recognised programming concepts and patterns, expressed using the Python programming language.

Encouragement! Many students jump to this exercise in preference to any other work we give you to do in the coming weeks. *Don't do this!* We provide *all* of these exercises, whether the non-assessed lab work, or this assessed exercise, in a *developmental sequence*, designed to help you develop proficiency progressively. The lab work is easier than this exercise. Do that first, to develop your skills – and then work on this exercise. You'll see notes on staging your work below.

The entire exercise targets the following valuable collection of concepts:

a. Core programming concepts such as expressions, variable creation and update, core control flow, files, the creation and use of functions, and use of libraries.
b. Textual input and output.
c. Object-oriented modelling of data and activity, including the use of classes, attributes, methods and inheritance
d. Event-driven graphical user interaction, including the use of functions as event handlers
e. Small algorithms, expressed in Python, whether created by you or created by someone else and adjusted to fit, to achieve sub-goals of your choosing
f. The structure of a whole application in the Model-View-Controller style

Connected assessment. In terms of your overall coursework load, this exercise is linked to a second activity, in the following manner. You will submit the exercise described here to be evaluated by us at the end of Week 9. In Week 10, we will run a second activity in the lab sessions: a lab test in which you will answer questions about your code and make small changes, submitting by typing answers and copy/pasting new or updated code into a Moodle quiz. We will assume completion of Phase 1.

Hence, deep understanding of the code you submit and how it connects to the task is very important.

One last point, before we get on. We always need to use a *vehicle* for you to demonstrate the skills listed above – by which we mean a specific *task* set in a specific *context*. There are great advantages to one common exercise for all students, but we appreciate that some of you may find the context and task more interesting or appealing than others. But remember, whether you are interested in the topic or not, your focus here is on demonstrating a high level of proficiency across the range of generic knowledge, skills and understanding that we hope you are developing. That is the important deliverable of the exercise.

Ok – so – let's discuss that context…

2. Description of the Context and Task

Overview

The task is to develop a game using Python that involves repairing a damaged ecosystem.

The game is played on an 8×8 board. All described game elements take up one complete square. The board contains healthy and sick forest areas. There are environmental stations that can supply eco-repair materials. And there are polluter characters that move around the board and cause pollution.

The player begins in the top-left corner of the board – call this (0,0). The player's goal is as follows:

- Collect eco-repair materials from the environmental repair stations (walking on to the station's square gives the player the repair materials)
- Take the repair materials to a sick forest area to repair it (walking on to a sick area with repair materials will repair it, using up the materials).

The player's journey is made complicated as follows:

- Fixed landscape features – mountains and lakes, through which the player cannot walk.
- The polluters, who are non-player characters (NPCs) that move around the board automatically - whenever they move on to a healthy forest area, it becomes sick.
- If a player moves on to a sick forest without any repair materials, or if a polluter moves onto the player, then the game is over.

In more detail

- The game is played on an 8×8 board, suggest using (0,0) to be the top left corner of the board.
- Initial layout has the following squares:
    - 4 mountains and 4 lakes
    - 5 healthy and 3 sick forests
    - 3 repair stations
    - 3 polluters and one player character.
- The player starts at coordinate (0,0) (top left).
- The player can carry only one bag of repair materials at a time
    - They collect this by walking on to a repair station
    - If they already are carrying unused repair materials, nothing happens
- The player repairs a sick forest when they walk on to it while carrying repair materials
    - The repair materials are used up, and the player scores a point for making the repair.
    - Walking on to a healthy forest does not use up the materials
- If the player repairs all sick forest squares, they gain a 2-point bonus.
- The game finishes immediately if
    - an NPC lands on the same square as the player, or vice versa.
    - a player moves on to a sick forest square without repair materials.
- The player initially has 20 turns to repair as many sick squares as possible.

3. Major Tasks

Phase 1: Working with Textual Input & Output (75%)

Part A – Board Setup (15 Marks)

The following is guidance on how you could set up the board, using a collection of classes that give you a *model* of the board. (As you will see later, this is an important part of the MVC pattern.)

Create classes to represent all the kinds of squares on the board. You'll probably want:

- A `Fixed` class – objects of which can represent any fixed features you want, such as the mountains and lakes described.
- A `RepairStation` class
- A `Forest` class – this can be sick or healthy
- An `NPC` class
- A `Player` class

All of these classes will have a board position, and a letter/code to be used when printing out the state of the board (and later, we can add an image, when it gets graphical). Can you see a use for *inheritance* here? A super-class `BoardItem`, and all the above classes inherit from it?

Create a class `Board` to represent an 8×8 game board. One approach to setting up the board is for the constructor to take a filename as parameter. The associated file contains data to populate the board. The file format is 8 rows of 8 comma separated values, using the following characters to represent the different kinds of board squares:

- M – Mountain
- L – Lake
- R – Repair Station
- SF – Sick Forest
- HF – Healthy Forest
- a dash, -, to represent an empty square

An example file in this format is:

```
-,-,-,-,HF,,-,HF
-,R,-,-,-,-,-,-
-,-,L,M,M,-,SF,-
-,HF,M,SF,-,-,-,-
-,M,-,L,-,-,R,-
-,-,-,L,L,-,-,-
-,R,-,-,-,-,HF,-
-,-,SF,HF,-,-,-,-
```

Inside the constructor function, you can read in this data, line by line, using `.split()` to break up the lines on the commas, and then use the resulting list to create the necessary objects to populate, for example, a list of lists that represents the board. You can use if…elif…elif… to test the items in the list that split produces, to decide which kind of object to create.

The file representation does not contain any NPCs or the player. You can use the random library to generate two random numbers for the (x,y) position of the NPCs. The position generated may be

directly on a fixed item, like a Mountain – so you need to set the code up to potentially generate a few positions until a free spot comes up. Loo up `random.randint()` as a way of generating the random numbers.

Create a player object at the top left corner of the grid.

Part B – Displaying the Board (5 Marks)

Add a method `printBoard` to your `Board` class to print the board state to screen, using symbols such as those found in the file format above (these symbols can be stored in your `BoardItem` objects. Until you create a fancy graphical interface, you can use this after every turn to display the board.

This simple textual display is the first version of the View part of the Model View Controller pattern.

You are now in a good position to test whether everything up to this point works. If you have developed all the above - creating a board from a file, adding NPCs and the player, and then printing the board out to see it has all worked - that is a major achievement – well done!

Part C – Player Movement (10 Marks)

You are now going to start developing a version of the Controller part of MVC. This is the bit that handles user input and decides what to do with it – getting updates happening in the Model according to the game logic, or updating the View.

To start with, we'll just use textual input.

In a new `controller.py` file, import the classes from your other file(s).

The player enters a direction by keying in a single character (u(p),d(own),l(eft),r(ight)) and a distance of up to 3 for the number of squares to move. Set up a **while** loop in the following form:

```
Get user input
while the user input isn't "q" (to quit):
        if user input == "u":
                # Deal with an up request
        elif user input == "d":
                # Deal with a down request
        # and so on for l and r

        distance = input( "… and the distance to travel? > " )
        # Now deal with the direction and distance
        …

        # Finally – be sure to get the next input from the user
        Get user input
```

Movement occurs square by square, stopping at board edges or Fixed objects.

Making the movement work well is interesting. You don't want to put the move code here in the *controller*. The controller needs to be telling the *model* to change the player's position. So the `Player` class should really have a `move` method.

See if you can get that set up. You could simplify the plan above and just send the movement letter (u, d, l, r) direct to the `move` method and let that method work out how to handle it!

When you have simple movement working, consider how you will address the impassable objects such as mountains and water and implement that aspect.

Part D – Polluter NPC Movement (10 Marks)

Each NPC moves a random distance of 1 to 3 squares in a random direction, one square at a time.

Part E – The RepairStation and repairing forests (10Marks)

Add in the repair materials attribute for the player, and ensure it is appropriately updated when a player moves on to a repair station, and that forests are appropriately repaired when moving on to a sick one with materials in hand.

Part F – Game over: after 20 turns and interactions with an NPC or sick forest (10 Marks)

If the turn limit is reached, then the score should be reported. The other two cases result in immediate game over – these should be reported with the score up to that point.

Part G – Intelligent Polluter NPC Movement (5 Marks)

Rather than random movement, create an intelligent polluter class with appropriate inheritance that moves more directly towards healthy forests or the player. Set up the game to have 1 of these and two of the original NPCs that move randomly.

Part H – Object-orientation, Program Structure and Functions (10 Marks)

Use suitable classes, inheritance, functions, meaningful variable names, internal documentation and appropriate decomposition.

Phase 2: Working with the Tkinter GUI (25%)

This phase involves upgrading the controller to use an interactive graphical user interface.

Part I– Drawing the board using images for the board pieces (10 Marks)

This first part gets you to draw the board using images for the board pieces. We will supply some images that you can use – but you can specialize your game with your own images.

At this stage, you can still use textual input to take in the movement requests from the user. Error messages can be written to the screen, using pop-up boxes, or in the window design you come up with, for example, for the turn count.

Part J – Mouse or keyboard GUI input for choosing player movements (15 Marks)

This final stage adds mouse-based interaction, with mouse clicks detected on the Tkinter window and mapped to the intended move position. Also using the keyboard. Advanced: use threads to have real-time movement of the player and the NPCs simultaneously.

Congratulations!! If you have got this far, you have created a full application in the MVC style. Great!

4. Submission Requirements

Submit all Python source code files required to run your solution, which we assume uses only libraries in the Python Standard Libraries. You **must not** use external libraries or we will not be able to evaluate your solution properly.